

Chapter 7. Multi-container Applications with Compose





Outline

- Describe compose and its common use cases.
- Configure a repeatable developer environment with Compose.



Orchestrate Containers with Podman Compose

- Many applications are developed **as microservices**.
- In the context of containers, **every microservice is a container image**, which developers manage as an independent container. Because one application is composed of **several containers**, it might become difficult to manage the containers as a group.
- **Podman Compose** is an **open source tool** that you can use to run **Compose files**.
- **A Compose file** is a **YAML file** that **specifies the containers to manage**, as well as the **dependencies among them**.



Orchestrate Containers with Podman Compose

```
version: '3'
services:
  orders: ❶
    image: quay.io/user/python-app ❷
    ports:
      - 3030:8080 ❸
    environment:
      ACCOUNTS_SERVICE: http://accounts ❹
```

- 1) Declare the orders container.
- 2) Use the python-app container image.
- 3) Bind the 3030 port on your host machine to the 8080 port within the container.
- 4) Pass the ACCOUNTS_SERVICE environment variable to the application.



Orchestrate Containers with Podman Compose

- Podman Compose **complies with** the **Compose Specification**, which is **used to define multi-container applications** for several runtimes, such as Podman.
- Although Compose files should work with any container runtime, some features might depend on specific implementations of the runtime.
- You might find **Compose files** called **docker-compose.yaml**.
- Docker introduced the docker-compose.yaml naming convention when it started the Compose project. However, **Docker open sourced the Compose Specification**, which also specifies the Compose file naming convention. According to the Compose Specification, the preferred name is **compose.yaml**.

Podman Compose became popular for the following uses



- **Development environments:** You can approximate a **production environment** on your **local machine** by providing applications, databases, or cache systems configuration in a single file. So, developers can **automate complex multi-container deployments** with one file.
- **Automated testing:** You can define several test suites, along with their configurations and environment variables, within the same file. In addition, you might approximate an automated pipeline environment on a single machine.
- **Using Podman Compose in a production environment is not recommended.** Podman Compose **does not support advanced functions** that you might need in a production environment, such as **load balancing, distributing containers to multiple nodes, managing containers on different nodes**, and others.
- If you need a **production container orchestration solution**, then **Kubernetes or Red Hat OpenShift** is a better option. Red Hat OpenShift can run **multi-container applications** on different host machines, or nodes.



Podman Pods

- Podman introduced an alternative **to orchestrate containers on a single host** with the usage of **Podman pods**.
- **Podman pods** can use and **produce Kubernetes YAML files**.
- These Kubernetes YAML files are a declarative **way to define and manage multiple containers and their resources**.
- Podman **generate kube tool** generates a **Kubernetes YAML file from existing Podman containers, pods, and volumes**.
- The **podman play kube tool reads** the Kubernetes YAML file and then **recreates the defined resources on a local machine**.
- These tools help to ensure that a container that is developed with Podman can easily migrate and work in a Kubernetes ecosystem.



The Compose File

- The **Compose file** is a **YAML file** that contains the following sections:
 - **version** (currently deprecated): Specifies the Compose version used.
 - **services**: Defines the containers used.
 - **networks**: Defines the networks used by the containers.
 - **volumes**: Specifies the volumes used by the containers.
 - **configs**: Specifies the configurations used by the containers.
 - **secrets**: Defines the secrets used by the containers.
- The **secrets** and **configs objects** are mounted as a file in the containers.



The Compose File

- The following Compose file **defines the backend and db services**. The terms service and container are used interchangeably. It also overrides the default command for the backend image by specifying the command property.

```
version: "3.9"
services:
  backend:
    image: quay.io/user/backend
    ports:
      - "8081:8080"
    command: sh -c "COMMAND"
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
```

- The backend keyword is **preceded by two spaces**; therefore the backend object is an attribute of the services object.



Start and Stop Containers with Podman Compose

- You can execute a Compose file by using the **podman-compose up** command, which creates the defined objects, **such as volumes or networks, and starts the defined containers.**
 - 1) Check whether the user_default network exists.
 - 2) Create the user_default Podman network.
- Podman generates predictable names for objects that are not explicitly named. In the preceding example, Podman creates a default network for the application.
- The default network naming convention **uses the current directory name and the _default suffix.** The preceding example contains the network **user_default** because it shows a Compose file executed in the /home/user home directory.
- The default container naming convention uses the DIRECTORY_SERVICE_NUMBER format. Because the preceding example does not configure the container name, it creates the user_db_1 container.
- The **podman-compose down** stop command stops running containers and remove containers that are defined as services.



Networking

- Use the **networks** keyword at the **same indentation level** as the **services** keyword to **create and use Podman networks**.
- If the networks keyword is **not defined** in the Podman Compose file, then **Podman Compose creates a default network that has DNS enabled**.
- Services in **different networks cannot interact**. In the following example, the **frontend service** cannot communicate directly with the **db service**.
- **Isolating network traffic** provides **security advantages**, because you **can prevent specific containers from accessing protected data**.

Networking



- **Compose file that declares three containers: a front-end application, a back-end application, and a database:**

```
version: "3.9"
services:
  frontend:
    image: quay.io/user/frontend
    networks: ❶
    - app-net
    ports:
    - "8082:8080"
  backend:
    image: quay.io/user/backend
    networks: ❷
    - app-net
    - db-net
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    networks: ❸
    - db-net

networks: ❹
  app-net: {}
  db-net: {}
```



Volumes

- You can declare volumes by using the **volumes** keyword. Then, mount the volumes in containers by using the **VOLUME_NAME:_CONTAINER_DIRECTORY_:_OPTIONS_** syntax.

```
version: "3.9"
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes: ❶
      - db-vol:/var/lib/postgresql/data

volumes: ❷
  db-vol: {}
```



Volumes

- If you created a volume that is **not managed by Podman Compose**, for example, because you used the **podman volume create** command, then you can specify it in the volumes definition.

```
version: "3.9"
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes:
      - my-volume:/var/lib/postgresql/data

volumes:
  my-volume:
    external: true
```

- If **external** is set to **true**: specifies that **this volume already exists on your host** and its **lifecycle is managed outside of that of the application**. Compose doesn't then create the volume, and returns an error if the volume doesn't exist.



Volumes

- You can also define **bind mounts** by providing **a relative or absolute path** on your host machine. In the following example, the **./local/redhat directory on the host machine is mounted** as the **/var/lib/postgresql/data directory within the container**.

```
version: "3.9"
services:
  db:
    image: registry.redhat.io/rhel8/postgresql-13
    environment:
      POSTGRESQL_ADMIN_PASSWORD: redhat
    ports:
      - "5432:5432"
    volumes:
      - ./local/redhat:/var/lib/postgresql/data:Z
```

Section 7.3. Build Developer Environments with Compose





Compose Overview

- Podman Compose is a community project that was created as an **alternative to Docker Compose**.
- Podman Compose translates Compose files, which are **YAML definitions for groups of containers** and their dependencies, into Podman CLI commands.
- Podman Compose interacts directly with Podman and **does not communicate with Podman's API socket**. In contrast, Docker Compose **communicates directly with the Docker daemon by using its REST API**.
- By **not interacting with the Podman's API socket**, Podman Compose removes the need to run the Podman service that provides the API socket, saving resource consumption and providing a more native and lightweight solution for Podman users.
- Additionally, because it interacts with Podman directly, **Podman Compose makes better use of rootless containers and pods than Docker Compose**.



Installing podman compose

- To use Podman Compose, a **compatible version of Podman** must first be installed on your system.
- **podman --version** is used to confirm podman version.
- **pip3 install podman-compose** is used to install podman compose as a Python package using pip3.
- **pip3 install <https://github.com/containers/podman-compose/archive/devel.tar.gz>** is used to install the latest development version from the Podman Compose GitHub repository.

Multi-container Developer Environments with Compose



- With Podman Compose you can place the required services into the Compose file inside your application repository so that developers can start these services by issuing a single **podman-compose up** command.
- Running your development environment dependencies like this isolates the developer from shared developer environments.
- Also, this predefined configuration **saves developers from having to locally configure services like databases or external dependencies**.
- For example, to develop a back-end application, you can use a single Compose file to deploy a development environment that includes a **PostgreSQL database** container and a **pgAdmin interface container** to manage the database.
- You can specify the service dependencies by using the **depends_on property** followed by a list of services that need to start first.

Multi-container Developer Environments with Compose



- The following example demonstrates that the **database-admin** container must **start after the database container**.

```
services:
  database:
    image: "registry.redhat.io/rhel9/postgresql-13"
    container_name: "appdev-postgresql"
    volumes:
      - ./scripts/database/initial-data:/opt/app-root/src/postgresql-start:Z
  ...output omitted...
  database-admin:
    image: "registry.connect.redhat.com/crunchydata/crunchy-pgadmin4:ubi8-4.30-1"
    container_name: "appdev-pgadmin"
    depends_on:
      - database
  ...output omitted...
```

Lab: Section 7.5: Multi-container Applications with Compose



Lab: Task

